Lesson 02 Demo 01
Building a CrewAI-Powered Agent with OpenAI

Objective: To demonstrate how to build a multi-agent system using CrewAI that intelligently identifies queries requiring real-time information and retrieves the most up-to-date data accordingly

Scenario: A product manager at a digital knowledge platform needs to manage an increasing volume of user queries, many of which require real-time updates and dynamic information. At present, these queries are reviewed and answered manually, leading to delays, inconsistencies, and rising operational costs. The goal is to automate the query-handling process using a multi-agent system that can identify when real-time data retrieval is needed and provide users with fast, accurate, and context-aware responses while maintaining efficiency and cost control.

Tools required: Visual Studio Code

Prerequisites: Azure OpenAI key

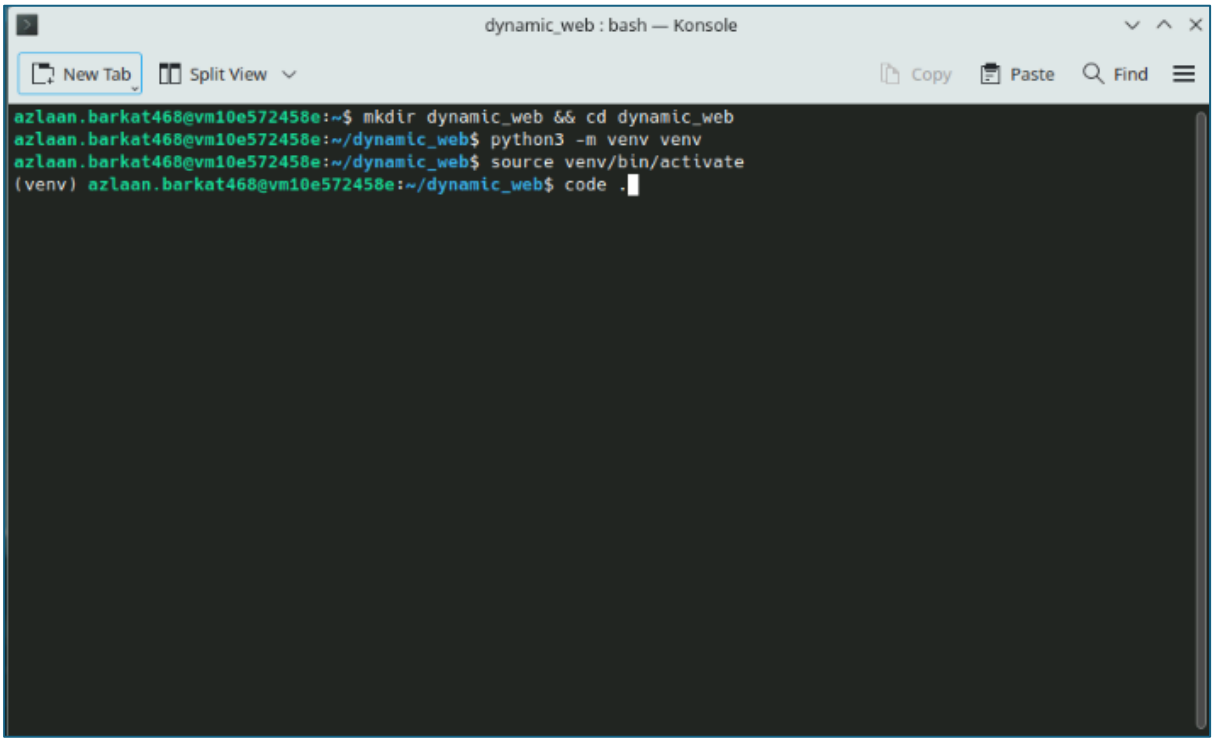
Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Configure the Tavily API Key and Azure OpenAI model
3. Define agents and tasks
4. Assemble and run the crew

Step 1: Set up your local environment in Visual Studio Code

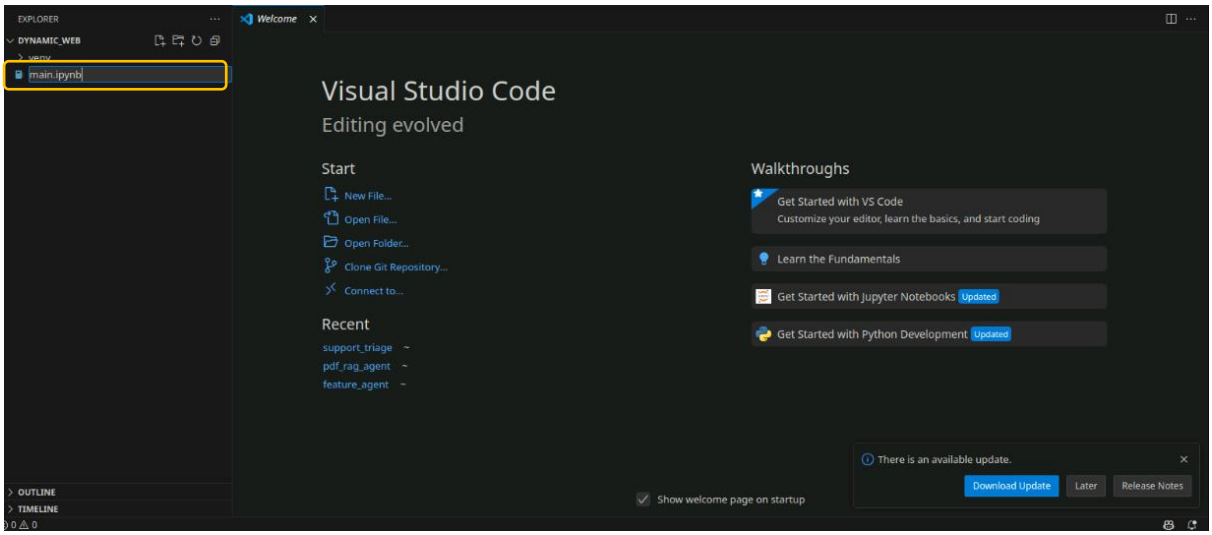
1.1 Open the terminal in your system and run the following commands:

mkdir dynamic_web && cd dynamic_web
python3 -m venv venv
source venv/bin/activate
code .



Note: This creates a virtual environment inside the dynamic_web folder and opens the project in VS Code.

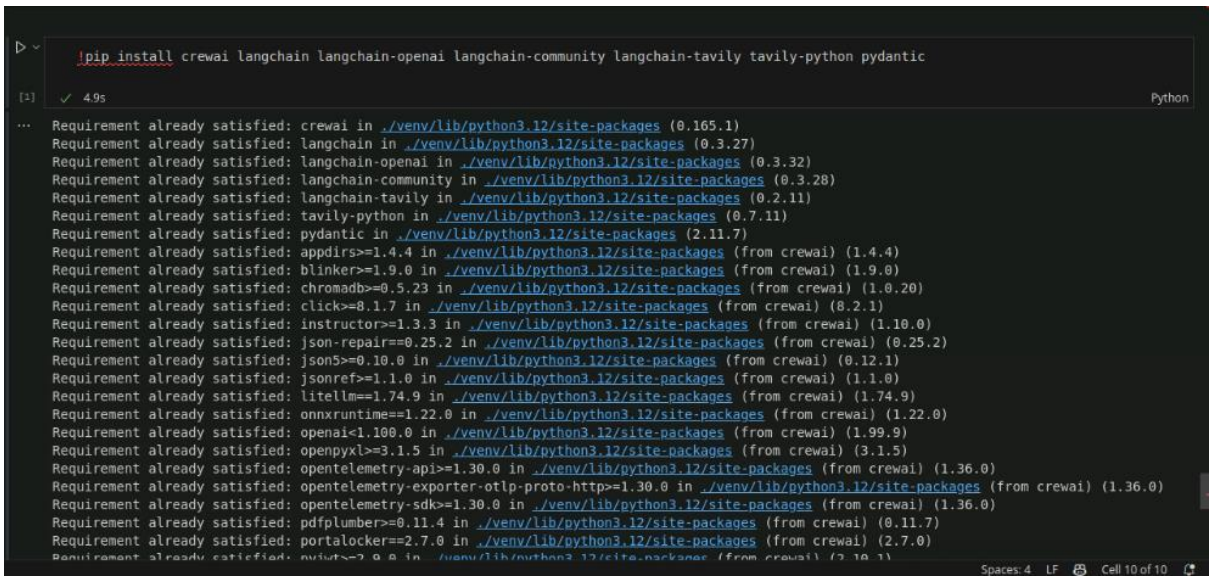
1.2 Create a new file and name it **main.ipynb**

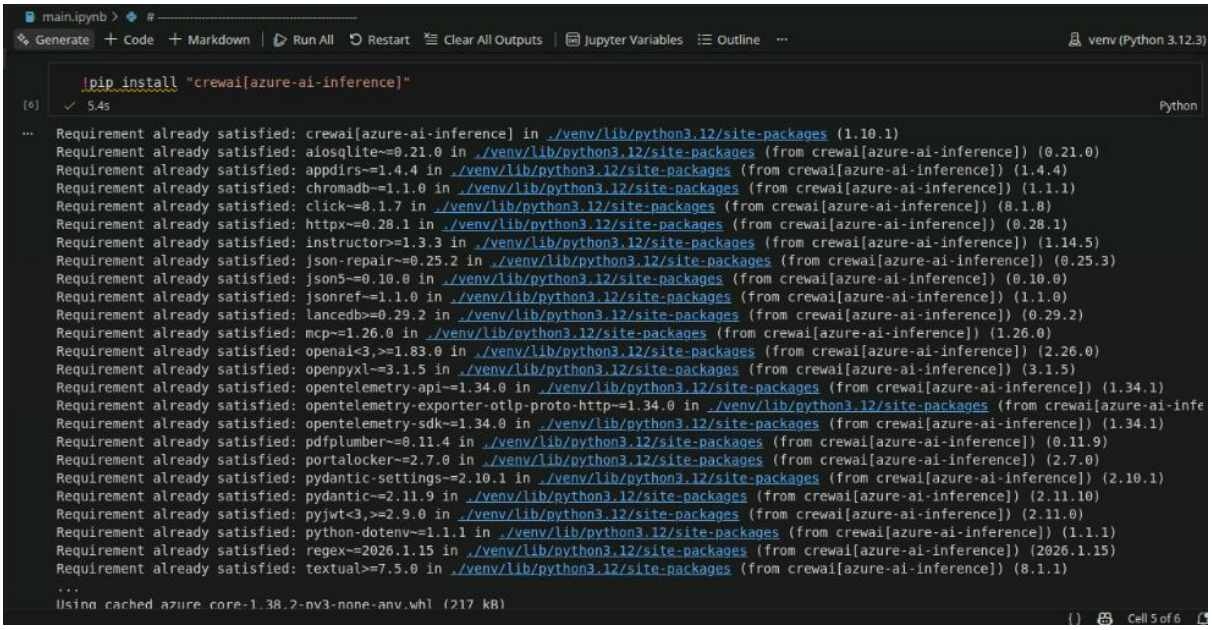


1.3 In the first 2 cells, write and run the following commands to install the required packages:

!pip install crewai langchain langchain-openai langchain-community langchain-tavily tavily-python pydantic

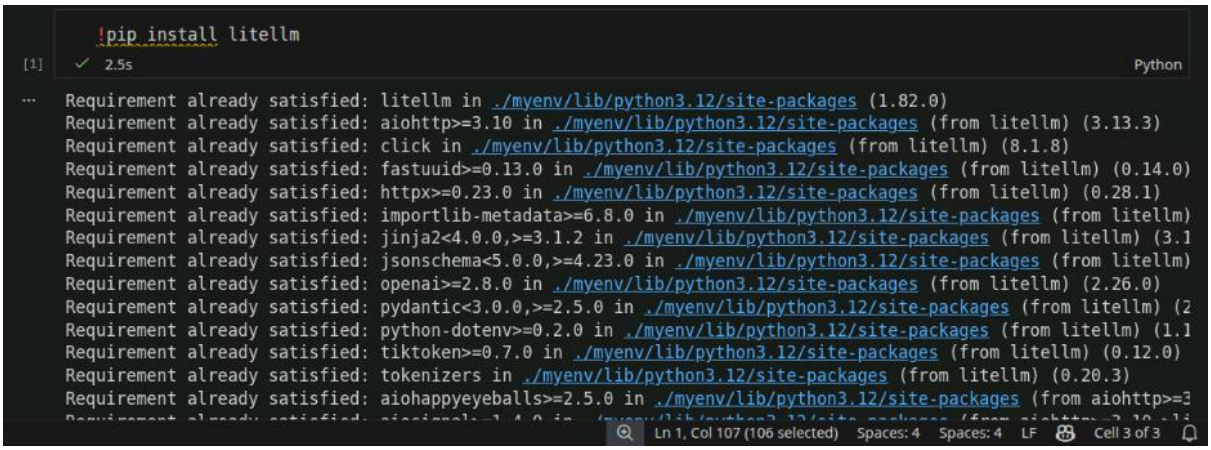
!pip install "crewai[azure-ai-inference]"





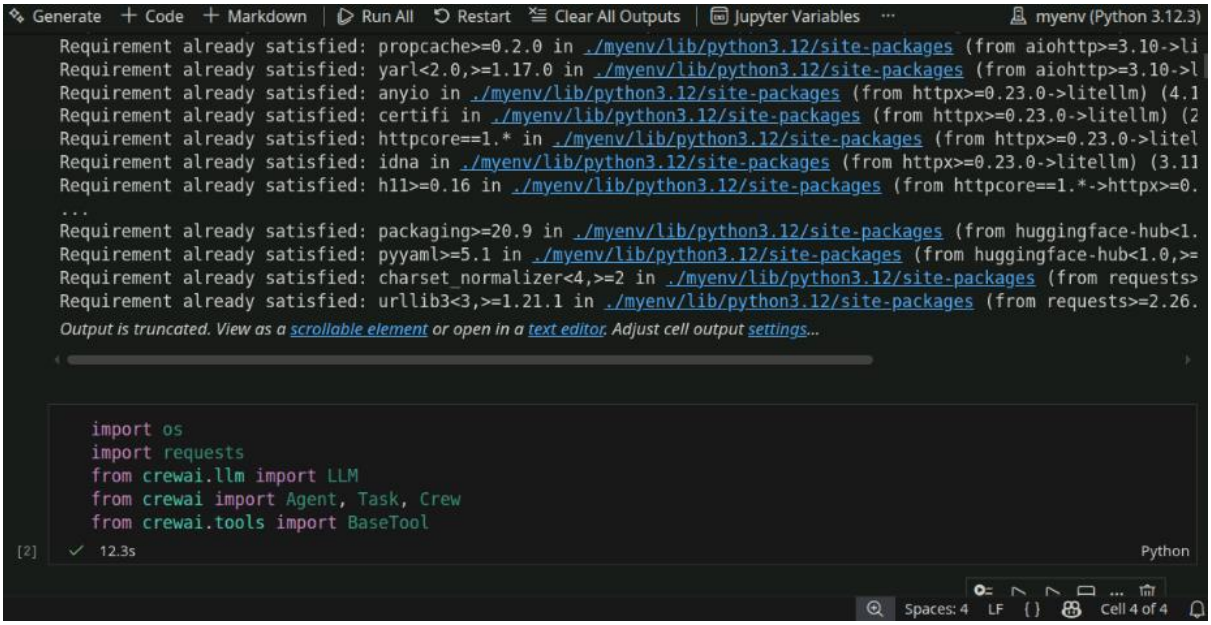
1.4 In the next cell, enter the following command to install litellm:

!pip install litellm



1.5 Import the required libraries using the following code:

import os
import requests
from crewai.llm import LLM
from crewai import Agent, Task, Crew
from crewai.tools import BaseTool



Step 2: Configure the Tavily API key and Azure OpenAI model

2.1 Use the Tavily API key to build a web search using the following code:

```
# Tavily API Key
TAVILY_API_KEY = "tvly-dev-1uT9rc-
X1i2A54uzUo18Hhz6BL1UjY8KMsU5aSCBbu3VRbNkr"

# ---- Custom CrewAI Tool for Web Search ----
class TavilySearchTool(BaseTool):
    name: str = "Web Search"
    description: str = "Search the web for recent information."

    def _run(self, query: str):
        url = "https://api.tavily.com/search"

        payload = {
            "api_key": TAVILY_API_KEY,
            "query": query,
            "max_results": 3
        }

        response = requests.post(url, json=payload)
        data = response.json()

        results = []
        for r in data["results"]:
```

```
        results.append(f"{r['title']} - {r['url']}")

    return "\n".join(results)


search_tool = TavilySearchTool()
```

```python
# Tavily API Key
TAVILY_API_KEY = "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"

# ---- Custom CrewAI Tool for Web Search ----
class TavilySearchTool(BaseTool):
    name: str = "Web Search"
    description: str = "Search the web for recent information."

    def _run(self, query: str):
        url = "https://api.tavily.com/search"

        payload = {
            "api_key": TAVILY_API_KEY,
            "query": query,
            "max_results": 3
        }

        response = requests.post(url, json=payload)
        data = response.json()

        results = []
        for r in data["results"]:
            results.append(f"{r['title']} - {r['url']}")

        return "\n".join(results)


search_tool = TavilySearchTool()
```

Note: Please create your own Tavily key and use it in the above code block.

2.2 Use the Azure OpenAI API key to build an LLM using the following code:

```
# ---- Azure LLM ----
llm = LLM(
   model="azure/gpt-5-mini",

api_key="2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99
BDAC77bzfXJ3w3AAABACOGi3sC",
   base_url="https://openai-api-management-gw.azure-api.net/",
   api_version="2024-12-01-preview",
   is_litellm=True,
   temperature=0,
   max_completion_tokens=3500
)
```

```
# ---- Azure LLM ----
llm = LLM(
    model="azure/gpt-5-mini",
    api_key="2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77bzfXJ3w3AAABACOGi3sC",
    base_url="https://openai-api-management-gw.azure-api.net/",
    api_version="2024-12-01-preview",
    is_litellm=True,
    temperature=0,
    max_completion_tokens=3000
)
```

Step 3: Define agents and tasks

3.1 Use the following code to create agents:

```
# ---- Agents ----
researcher = Agent(
   role="AI Researcher",
   goal="Find the latest advancements in AI for FMCG using web search",
   backstory="Expert in AI research trends who gathers real-time information from
the web.",
   verbose=True,
    allow_delegation=False,
   llm=llm,
   max_iter=1,
   tools=[search_tool]
)

writer = Agent(
   role="Technical Writer",
   goal="Summarize research into an executive report",
   backstory="Expert in executive summaries.",
   verbose=True,
   allow_delegation=False,
   llm=llm,
   tools=[search_tool]
)
```

```python
# Researcher agent
researcher = Agent(
    role="AI Researcher",
    goal="Find the latest advancements in AI for FMCG",
    backstory="You are an expert in artificial intelligence and stay updated with the latest research trends in FMCG.",
    verbose=True,
    allow_delegation=False,
    llm=llm,
    max_iter=1,
    tools=[search_tool]
)

# Writer agent
writer = Agent(
    role="Technical Writer",
    goal="Summarize research into an executive report",
    backstory="You are an experienced technical writer with expertise in summarizing research for executives.",
    verbose=True,
    allow_delegation=False,
    llm=llm,
    tools=[search_tool]
)
```

3.2 Define the research and writing tasks:

```
# ---- Tasks ----
task_research = Task(
   description="Search the web and identify the top 3 recent advancements in AI for FMCG.",
   expected_output="Detailed notes explaining three recent AI advancements in FMCG with examples.",
   agent=researcher
)

task_write = Task(
   description="""
Write a concise executive summary using the research notes.

Requirements:
- Maximum 100 words
- Use bullet points
- Focus only on the 3 key advancements
""",
   expected_output="Executive summary of AI advancements in FMCG.",
   agent=writer,
   context=[task_research]
)
```

```
# ---- Tasks ----
task_research = Task(
    description="Search the web and identify the top 3 recent advancements in AI for FMCG.",
    expected_output="Detailed notes explaining three recent AI advancements in FMCG with examples.",
    agent=researcher
)

task_write = Task(
    description="""
Write a concise executive summary using the research notes.

Requirements:
- Maximum 100 words
- Use bullet points
- Focus only on the 3 key advancements
""",
    expected_output="Executive summary of AI advancements in FMCG.",
    agent=writer,
    context=[task_research]
)
```

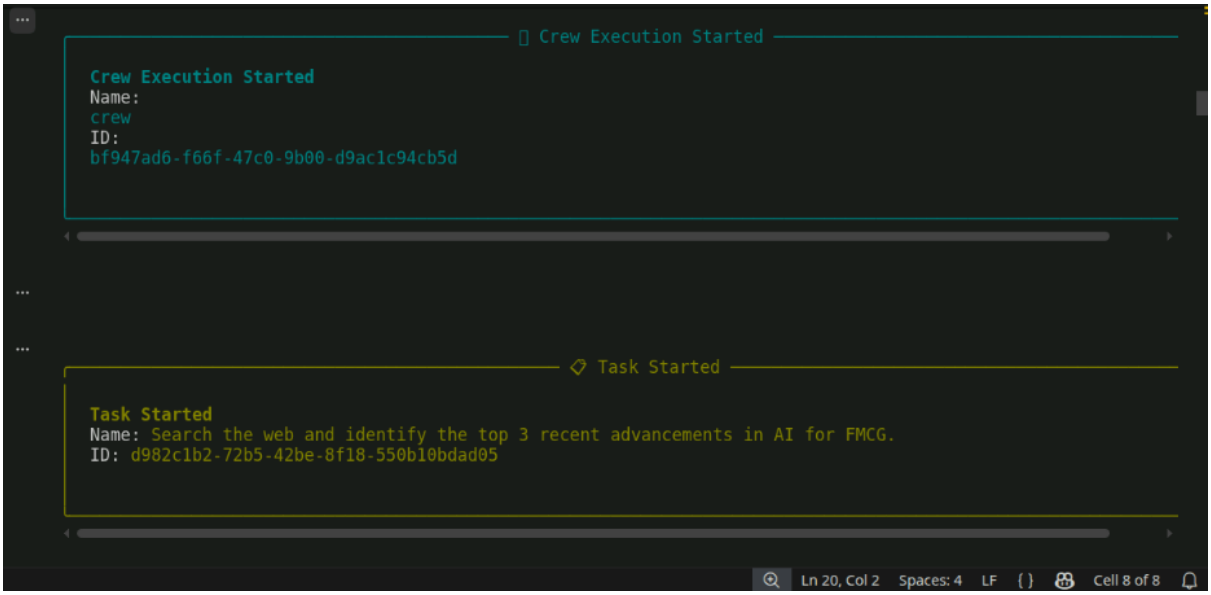
Step 4: Assemble and run the crew

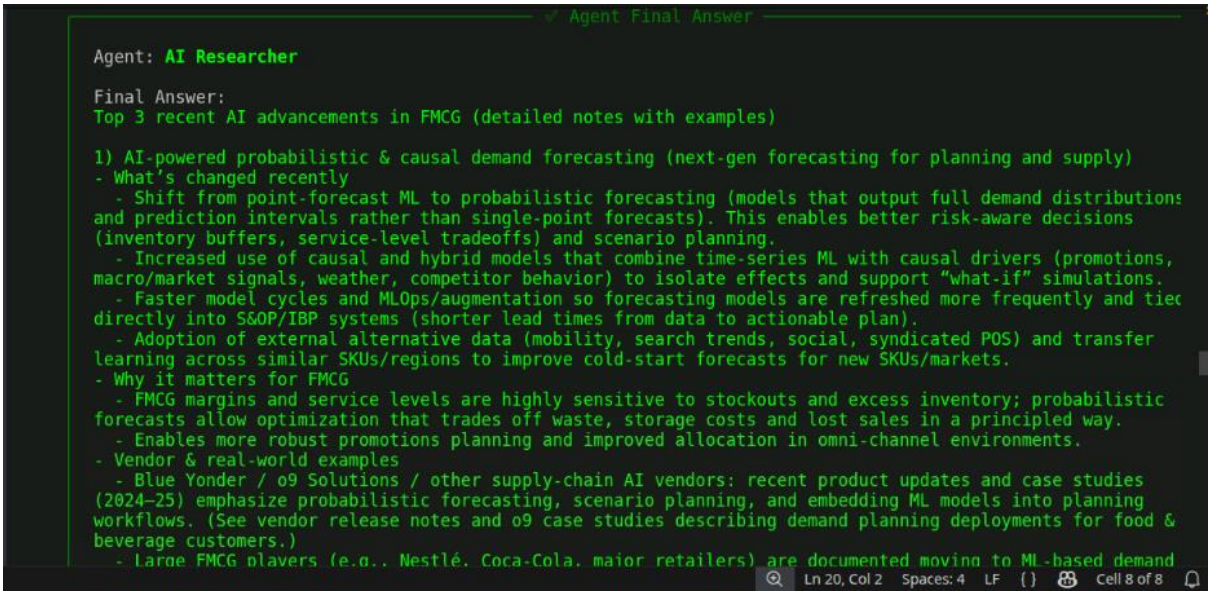
4.1 Use the code given below to arrange all the components of the crew and run it:

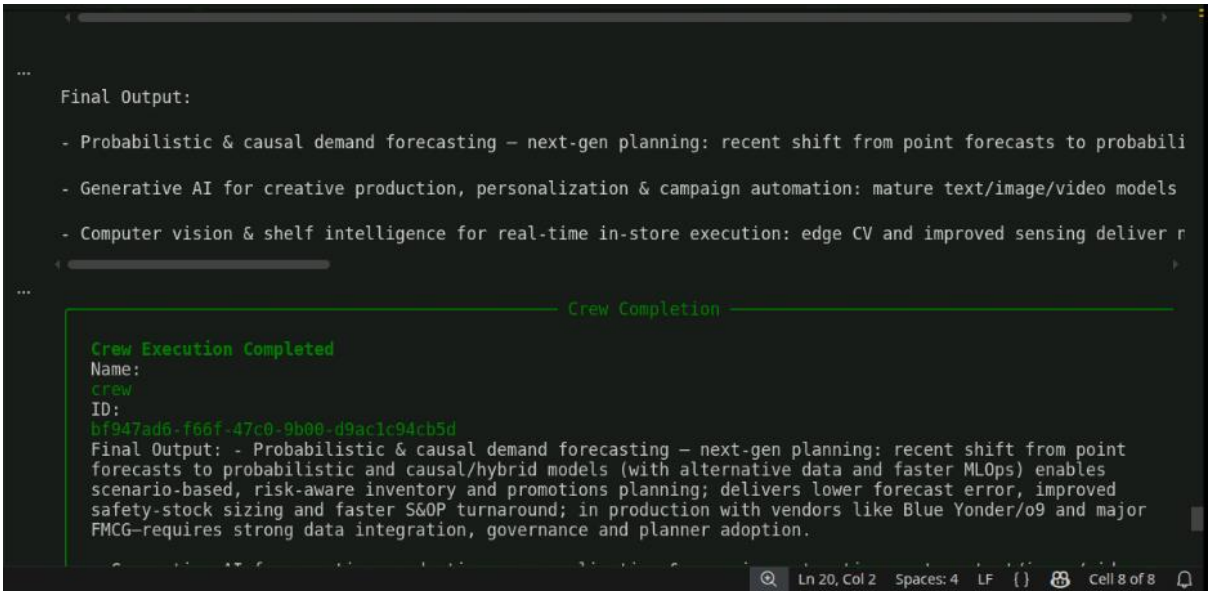
```
# ---- Crew ----
crew = Crew(
   agents=[researcher, writer],
   tasks=[task_research, task_write],
   verbose=True
)

result = await crew.kickoff_async()

print("\nFinal Output:\n")
print(result.raw)
```

```
# ---- Crew ----
crew = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)

result = await crew.kickoff_async()

print("\nFinal Output:\n")
print(result.raw)
```

The program starts generating a summary report, as shown in the following screenshots:







By following these steps, you have successfully built a CrewAI-powered multi-agent system that intelligently routes and processes user queries, determines when real-time web data is needed, and retrieves the latest results via the Tavily Search Tool.